Полиморфизм

> Полиморфизм – способность объекта по одному и тому же интерфейсу отвечать по-разному, в зависимости от реализации.[1]

> Полиморфизм – Означает, что схожие объекты способны по-разному отвечать на одно и то же сообщение.[2]

В отличие от классических статически типизированных языков, в Python полиморфизм гораздо шире и проникает на все уровни языка — от классического наследования до динамических протоколов

Классический полиморфизм через наследование и переопределение методов

В традиционном ООП родительский класс определяет общий интерфейс (например, метод `draw()`), а дочерние классы переопределяют этот метод своей конкретной реализацией

```python
class Shape:
    def draw(self) -> None:
        pass

class Circle(Shape):
    def draw(self) -> None:
        print("Рисуем круг")

class Rectangle(Shape):
    def draw(self) -> None:
        print("Рисуем прямоугольник")
```

Единый интерфейс, но разное поведение объектов `shapes`:

```python
shapes: list[Shape] = [Circle(), Rectangle()]
for shape in shapes:
    shape.draw()
```

```
Рисуем круг
Рисуем прямоугольник
```

Утиная типизация

В Python наследование от общего базового класса вовсе не обязательно. Главный принцип динамической утиной типизации гласит: если объект поддерживает необходимый метод или операцию, не имеет значения, к какому классу он принадлежит.

```python
class Document:
    def render(self) -> None:
```

[1] определение из лекции по ООП от 10.10.26
[2] Объектно-ориентированное мышление – Мэтт Вайсфельд

```
        print("Рендеринг документа...")

class Image:
    def render(self) -> None:
        print("Отображение картинки...")

def display(printable_object) -> None:
    printable_object.render()
```

И такие вызовы работают без общего родительского класса:

```
display(Document())
```
Python

```
Рендеринг документа...
```

```
display(Image())
```
Python

```
Отображение картинки...
```

Среда выполнения Python проверяет наличие и поддержку операций объектом непосредственно в момент выполнения

Статическая утиная типизация

Чтобы сохранить гибкость утиной типизации и при этом дать возможность статическим анализаторам (таким как Муру) проверять типы до запуска программы, в Python используются протоколы из модуля `typing`

```
from typing import Protocol

class Renderable(Protocol):
    def render(self) -> None: ...

class WebPage:
    def render(self) -> None:
        print("Отображение веб-страницы")

def render_element(element: Renderable) -> None:
    element.render()

render_element(WebPage())
```
Python

```
Отображение веб-страницы
```

Классу не требуется явно объявлять наследование от протокола — достаточно просто реализовать совпадающие методы

Гусиная типизация

Когда необходимо явно зафиксировать контракт интерфейса во время выполнения, используется проверка принадлежности к абстрактным базовым классам (ABC) с помощью

```
from collections.abc import Iterable
```
Python

```
def process_items(items: Iterable) -> None:
    if isinstance(items, Iterable):
        for item in items:
            print(item)
```

Проверка через ABC разрешает полиморфизм для любых итерируемых объектов

Это позволяет проверять соответствие интерфейсу, сохраняя полиморфизм для любых совместимых классов или зарегистрированных виртуальных подклассов

Одиночная диспетчеризация

Полиморфизм на уровне отдельных функций позволяет выполнять разный код в зависимости от типа переданного аргумента во время выполнения

```python
from functools import singledispatch

@singledispatch
def process(data) -> None:
    print(f"Базовая обработка: {data}")

@process.register(int)
def _(data: int) -> None:
    print(f"Числовая обработка: {data * 2}")

@process.register(list)
def _(data: list) -> None:
    print(f"Список из {len(data)} элементов")

process("текст")
```

```
Базовая обработка: текст
```

```python
process(10)
```

```
Числовая обработка: 20
```

```python
process([23, 24])
```

```
Список из 2 элементов
```

Такой подход облегчает модульное расширение программы без раздувания цепочек `if/elif` или изменения существующих классов